

ВОДОКАНАЛ · AI-ПОМОЩНИК АБОНЕНТА

Сдача проекта

AI-помощник абонента водоканала

Консультации и подготовка обращений в личном кабинете — что построено на Этапе 1 и как это работает

Прототип · демо-стенд

24 сентября 2026

Абонент теряет время там, где мог бы получить ответ сразу



Не знает, какое обращение выбрать

Десятки типов обращений — не всегда понятно, какой подходит под конкретную ситуацию



Ошибается при заполнении

Обращение с недостающими данными возвращается на доработку или обрабатывается дольше



Не понимает, что дальше

После отправки обращения не ясно, чего ждать и когда — отсюда повторные обращения по тому же вопросу



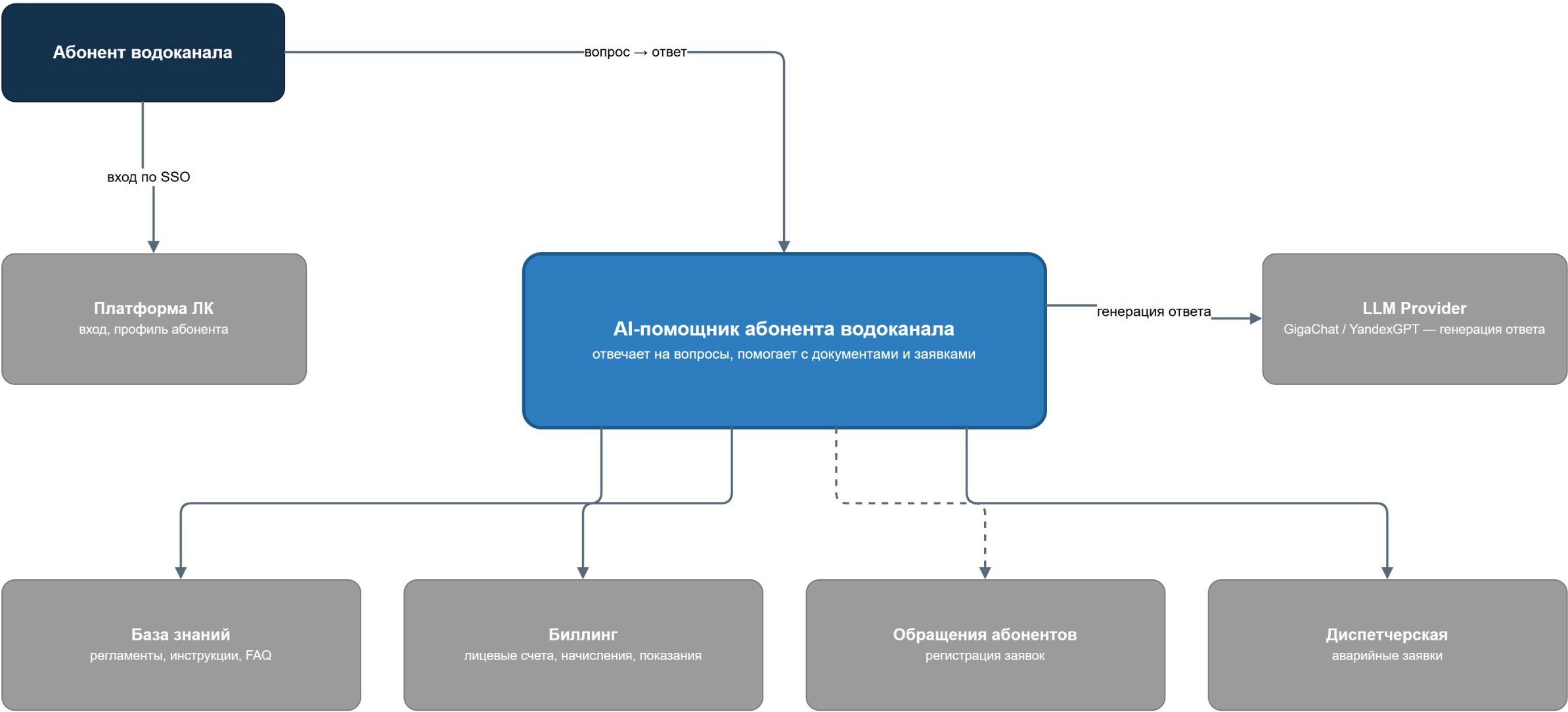
Звонит в колл-центр

Каждый типовой вопрос без самостоятельного ответа — лишняя нагрузка на операторов и очередь для всех

Что должно измениться для абонента и для водоканала

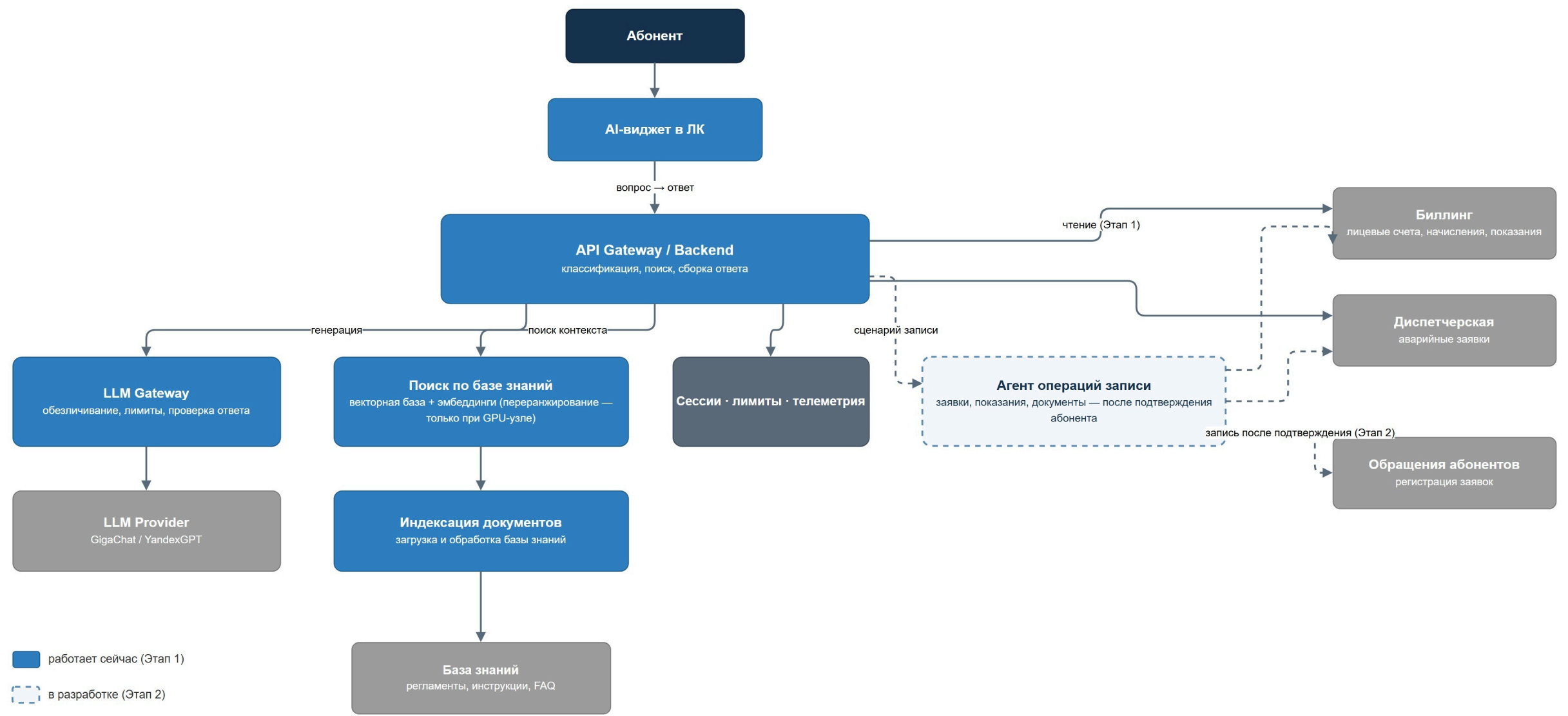
- ✓ Сократить время поиска ответа на вопрос
- ✓ Снизить долю обращений, оформленных с ошибками
- ✓ Снизить число повторных обращений по одному вопросу
- ✓ Улучшить качество взаимодействия абонента с личным кабинетом
- ✓ Снизить нагрузку на операторов колл-центра за счёт самообслуживания

С кем взаимодействует помощник



- - - регистрация заявок — Этап 2, в разработке

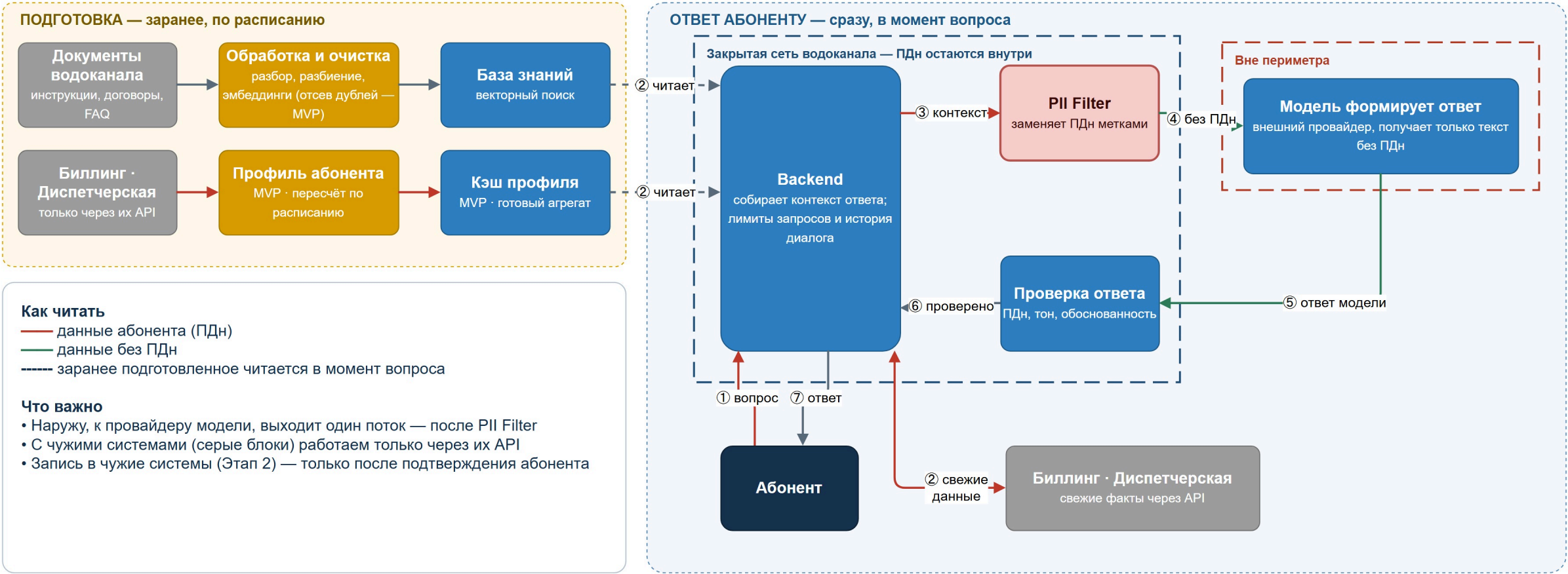
Из каких частей состоит решение



Контейнеры: на чём сделаны, почему так, что на MVP

Блок	На чём сделан	Почему так	Что будет на MVP
AI-виджет в ЛК	Сейчас HTML + JavaScript без фреймворка, поток ответа — SSE. На MVP — React / TypeScript.	Встраивается в существующий личный кабинет; ответ приходит потоком, токен за токеном.	Образ ai-widjet за Ingress (nginx), 2 реплики; вход по SSO личного кабинета.
API Gateway / Backend	Python 3.12, FastAPI, uvicorn, pydantic; правила классификации на регулярных выражениях.	Один язык с моделями и обезличиванием. Замер: 117 запросов/с на процесс, 8,5 мс CPU на запрос без модели.	Deployment, 2 реплики (3 на тираже) с автомасштабированием; сессия привязывается к SSO — без этого запись от имени абонента включать нельзя.
LLM Gateway	Python, LiteLLM Router, Presidio + spaCy, Redis (лимиты).	Единый обязательный путь к модели: лимиты → обезличивание → модель → проверка; обхода быть не может. Переключение на запасного провайдера проверено на живом трафике.	Отдельный Deployment, 2 реплики. Основной провайдер — по проверке на живых данных: предварительно GigaChat Pro, YandexGPT — резерв.
LLM Provider	Управляемый API в РФ: YandexGPT Pro (сейчас), GigaChat Pro.	Данные остаются в РФ (152-ФЗ), собственный GPU не нужен. Модель получает только обезличенный текст. Своя модель (Qwen2.5-32B) — только по триггеру: запрет ИБ или нагрузка.	Два провайдера через шлюз; YandexGPT: до первого куска ответа медиана 0,78 с, P90 5,6 с (19 вызовов).
Поиск по базе знаний	sentence-transformers, e5-small на CPU (22–37 мс), Qdrant (HNSW), клиент по gRPC.	e5-small укладывается в бюджет поиска 100 мс на CPU; BGE-M3 (210–300 мс) и ранкер (7,2 с) — нет. gRPC стабилен: 6–13 мс против разброса 6–60 мс по REST.	Qdrant в Kubernetes (StatefulSet, диск NVMe); эмбединги на CPU, ранкер выключен. GPU-узел — только по решению владельца после проверки на реальных данных.
Индексация документов	Python: python-docx → части до 300 знаков → проверка ПДн (Presidio) → эмбединги → Qdrant.	Отдельный прогон, а не при старте Backend. Новая версия индекса собирается рядом, алиас переключается атомарно — поиск не останавливается.	Kubernetes CronJob, окно 02:00–05:00, жёсткая граница 3 часа. Объём: 3 000–5 000 документов, 30–100 тыс. частей (допущение).
Сессии · лимиты · телеметрия	Redis 7 (сессии с TTL 30 минут, лимиты), PostgreSQL 16 (обращения, телеметрия), psycorp 3.	Redis даёт структуры для сессии и счётчиков; Postgres — материализованные представления. Отдельная база под телеметрию не нужна: объём мал (замер: 12 млн строк — 3,5 ГБ).	StatefulSet с постоянными томами, резервные копии по расписанию в S3 / MinIO.
Агент операций записи и связанные контейнеры	Python, собственный конечный автомат (7 состояний); Billing Adapter (XML-RPC), сервис обращений, генератор документов (PDF-библиотека не выбрана).	Запись возможна только из состояния «ожидает подтверждения абонента», ни одна запись без подтверждения. LangGraph — по условию, не сейчас (ADR-200).	Разворачиваются на MVP после проверки пользы и безопасности Этапа 1 (stage-gating); PDF хранятся в S3 / MinIO.

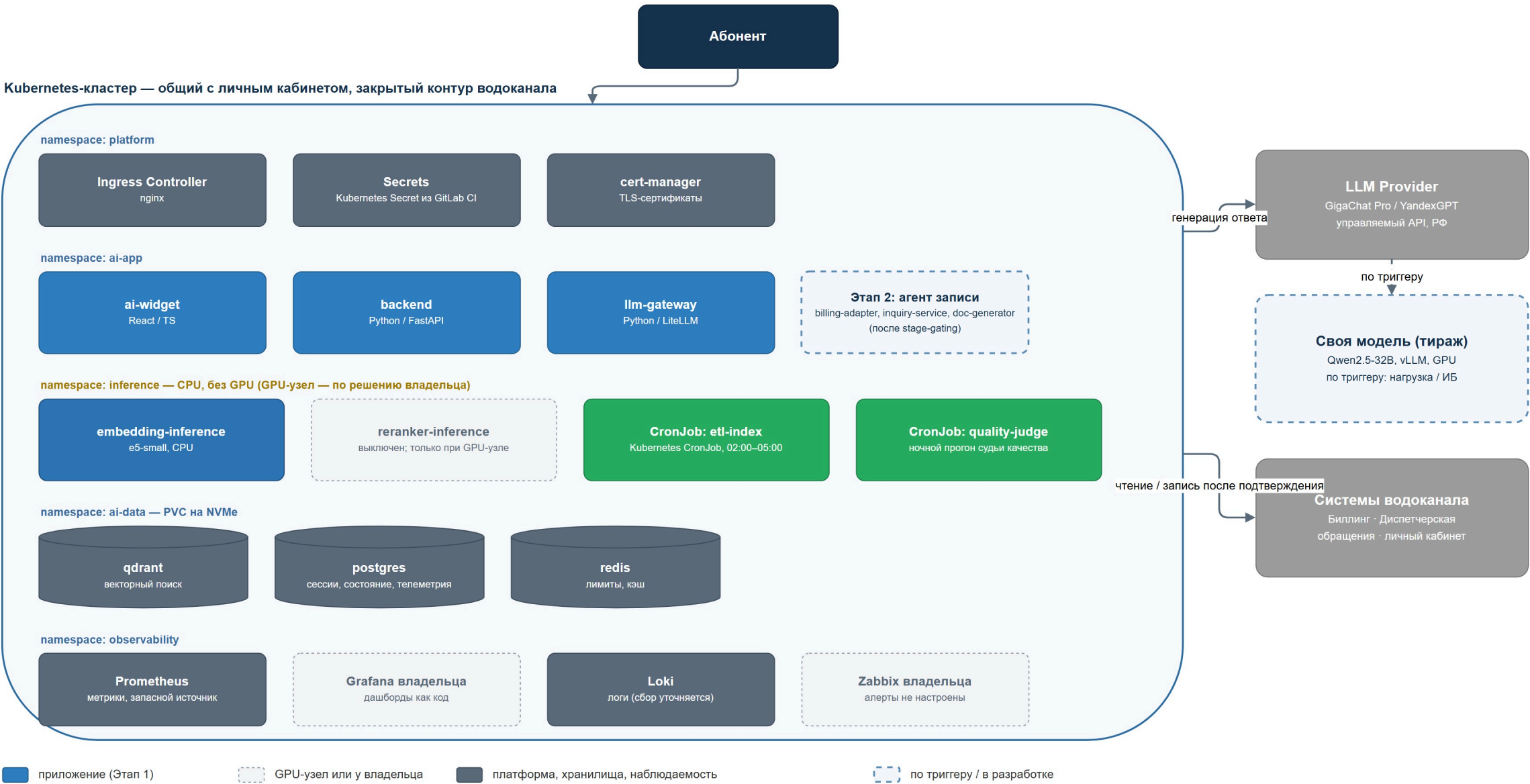
Как данные становятся ответом абоненту



Поток данных: на чём сделан каждый шаг, почему так, что на MVP

Блок	На чём сделан	Почему так	Что будет на MVP
Обработка и очистка документов	python-docx (Word), нарезка на части, Presidio, эмбединги e5-small на CPU.	Персональные данные в корпусе — ошибка сборки: в индекс они не попадают. Дубли отсеиваются, чтобы не размывать выдачу.	CronJob по расписанию. Отсев дублей и офлайн-реранкинг — работа MVP: в коде их пока нет.
База знаний (векторный поиск)	Qdrant: индекс HNSW (m=16, ef=128), фильтр по типу обращения внутри поиска.	Фильтр внутри поиска не теряет нужный фрагмент из-за соседей другого типа. Замер: 100 000 векторов — 477 МБ на диске, поиск 6–13 мс по gRPC.	30–100 тыс. векторов; топ-20 кандидатов, в промпт идут 3 лучших фрагмента; порог близости подобран замером.
Профиль абонента и его кэш	Kubernetes CronJob → материализованное представление PostgreSQL (без Redis).	Ответ читает готовый агрегат по индексу и не обращается к первичным системам; свежие факты Backend по-прежнему берёт через API.	Это работа MVP: на прототипе профиль читается из слепка данных, пересчёта по расписанию нет.
Backend	Python, FastAPI; правила выбирают путь ответа, модель зовётся не всегда.	Часть вопросов закрывают правила и данные абонента без модели (замер: до модели доходит 30–70% вопросов): быстрее и дешевле.	2 реплики, лимиты и история диалога в Redis, привязка сессии к SSO.
PII Filter	Presidio + spaCy (ru_core_news_md) + собственные распознаватели: СНИЛС, ИНН, лицевой счёт, телефон, e-mail.	Для 152-ФЗ пропуск персональных данных опаснее ложной находки. Замерены только ложные находки на 136 обращениях, полнота — нет.	Замер полноты и точности на живых данных (4–6 чел-дн), выбор модели spaCy по результату.
Модель формирует ответ	Внешний провайдер в РФ через LiteLLM: YandexGPT сейчас, на MVP предварительно GigaChat Pro основной.	Наружу уходит один поток и только после обезличивания; ответ по найденным фрагментам, с источниками и честным отказом при низкой уверенности.	Основной и запасной провайдер — по проверке на живых данных; выбор фиксируется в ADR-300.
Проверка ответа	Регулярные выражения на потоке: ПДн, тон, политика, фразы о невыполненном действии; судья качества — отдельной моделью после ответа.	Быстрая проверка (бюджет 50 мс на кусок) блокирует, судья — только оценивает. Замер: 800 знаков — P95 ≈49 мс, 2 000 знаков — ≈120 мс (выше бюджета).	Оптимизация: проверять новый кусок с небольшим перекрытием, а не весь накопленный текст; настройка порогов на живых ответах.
Биллинг и Диспетчерская	XML-RPC API Биллинга, REST API Диспетчерской; у нас только чтение.	Единственный разрешённый способ доступа к чужим системам: своей копии чужих данных не держим.	Чтение по живым API вместо слепков прототипа; запись — Этап 2 и только после подтверждения абонента.

Где и на чём развёрнута система на MVP

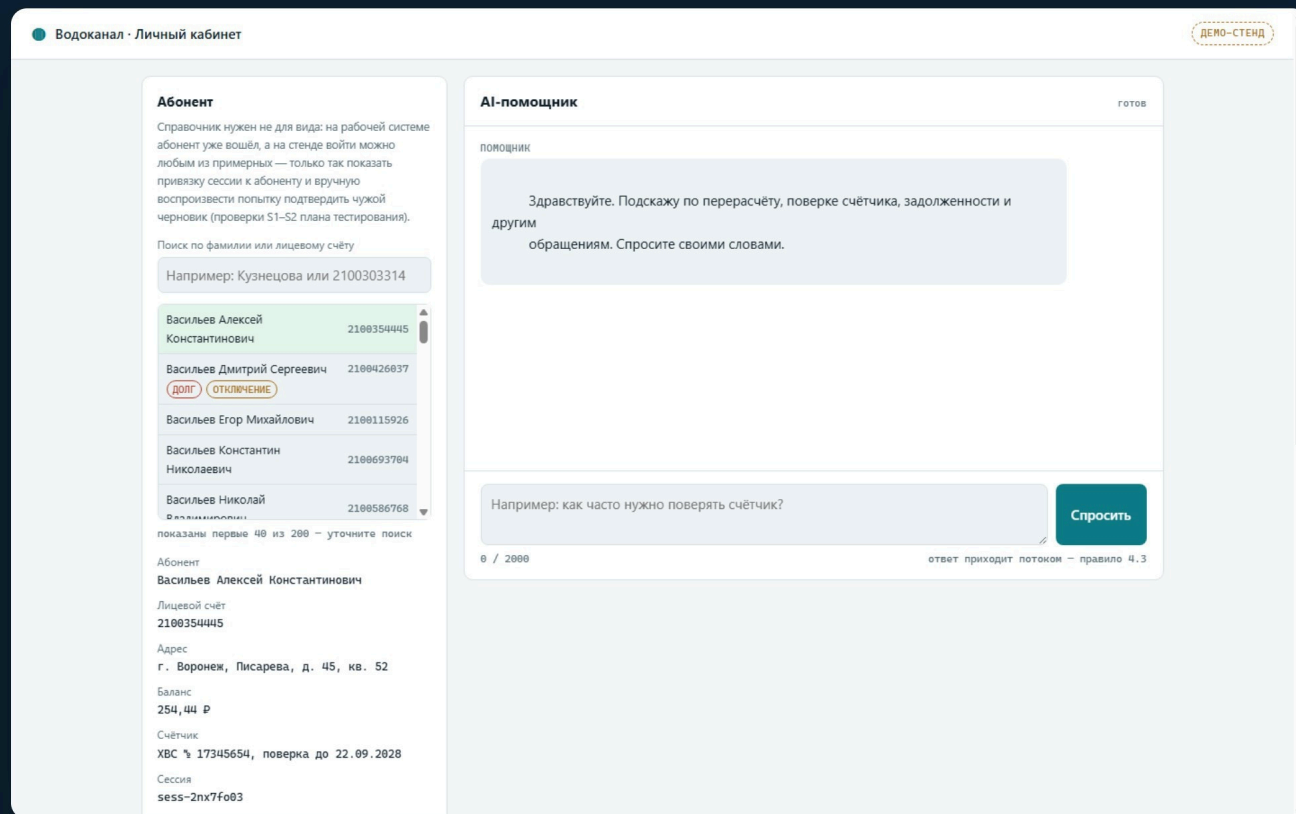


Развёртывание: на чём сделано, почему так, что на MVP

Блок	На чём сделан	Почему так	Что будет на MVP
Платформа: Kubernetes	Общий кластер Kubernetes, где уже работает личный кабинет; эксплуатирует тот же отдел.	Платформа уже есть; нужны отказоустойчивость, обновление без остановки, задания по расписанию. Нагрузка ради этого не нужна: запросы подов помещаются на один сервер.	По расчёту 6 узлов (4 общих, 2 данных) — верхняя оценка для общего кластера; GPU-узлов нет; дистрибутив, квоты и правила размещения — вопросы владельцу.
Ingress, сертификаты, секреты	nginx Ingress, cert-manager; секреты — GitLab CI variables → Kubernetes Secret.	GitLab уже используется, ничего нового ставить не нужно; приложение читает переменные окружения, как на прототипе. Vault — по условиям пересмотра (аудит чтения, динамические секреты).	Переменные masked и protected, деплой через pipeline; 2–3 чел-дн на настройку и проверку, что значения не попадают в логи.
ai-app: виджет, backend, шлюз	Deployment для ai-widget, backend, llm-gateway; автомасштабирование, запас на потерю пода.	Stateless-контейнеры: состояние вынесено в Redis и Postgres, поэтому реплики можно добавлять и перезапускать.	По 2 реплики (backend — 3 на тираже); вместе с контейнерами Этапа 2 (агент, Billing Adapter, сервис обращений, генератор документов) после stage-gating.
inference: эмбединги и задания	embedding-inference (e5-small, CPU, 2 реплики); CronJob etl-index и quality-judge; reranker-inference выключен.	В кластере нет GPU: BGE-M3 и реранкер на CPU не укладываются в бюджет. Стоит ли покупать GPU, решает тест на реальных данных; сначала CPU + YandexGPT.	Тест на реальных данных; аренда GPU на месяц — только если CPU + YandexGPT не подойдёт; покупка — по итогам аренды.
ai-data: хранилища	Qdrant, PostgreSQL 16, Redis — StatefulSet с постоянными томами на NVMe.	Состояние и счётчики должны переживать перезапуск пода; индекс и телеметрия — на быстром диске.	≈170 ГиБ дисков на MVP (расчёт); резервные копии CronJob раз в сутки в S3 / MinIO.
observability: мониторинг	Zabbix и Grafana владельца; Prometheus и Loki — запасные; метрики нашего сервиса — на порту 9100.	Мониторинг и Grafana уже есть в контуре, дублировать не нужно. Риск: считать P99 по гистограммам Zabbix может неудобно — проверяется до работ.	Дашборды и 9 алертов как код; алерты пока не настроены: получатели и каналы определяются до включения MVP.
Внешние сервисы	LLM Provider: управляемый API в РФ (YandexGPT Pro, GigaChat Pro); системы водоканала — только через их API.	Данные абонента не покидают контур без обезличивания; своей копии чужих данных нет.	Собственная модель Qwen2.5-32B на vLLM с GPU A100 80 ГБ — не на MVP, только на тираже по триггеру.

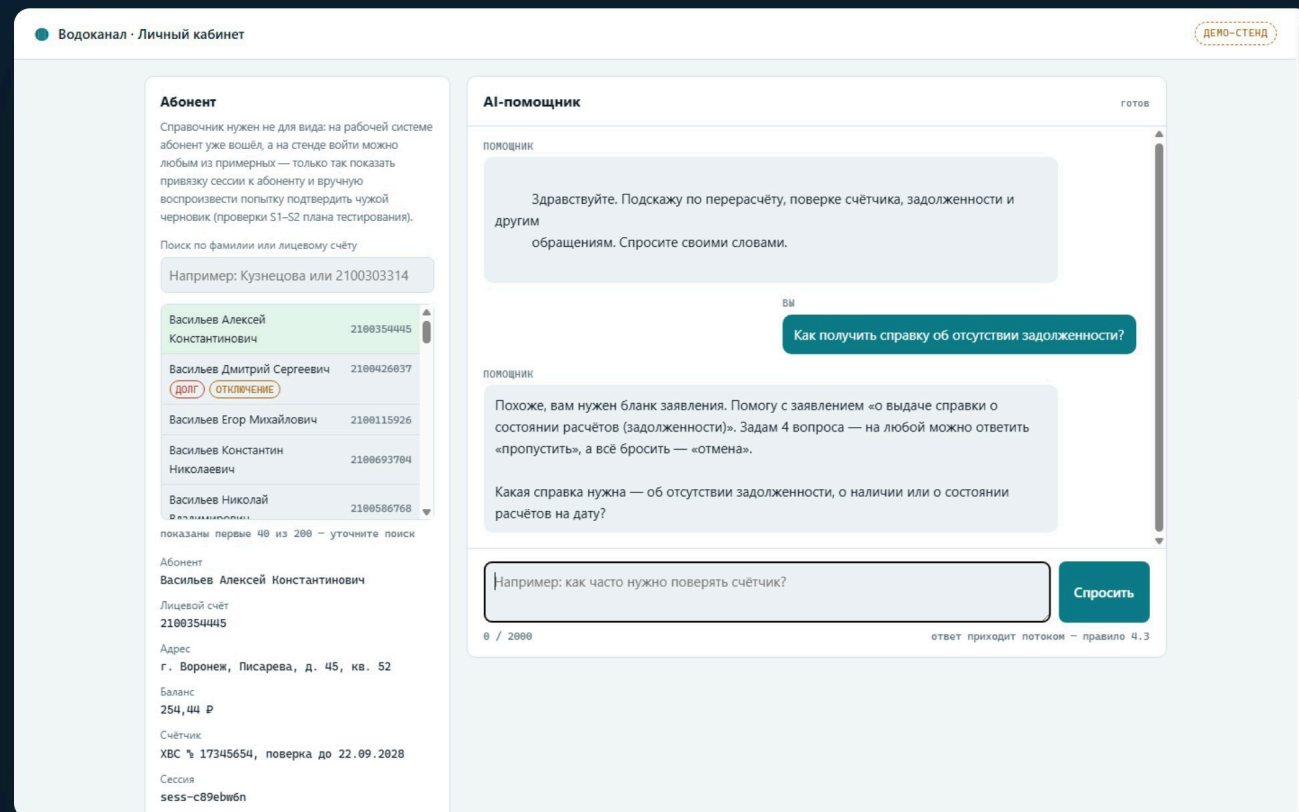
КАК ЭТО ВЫГЛЯДИТ В ИНТЕРФЕЙСЕ

Демо-стенд: карточка абонента и диалог с помощником



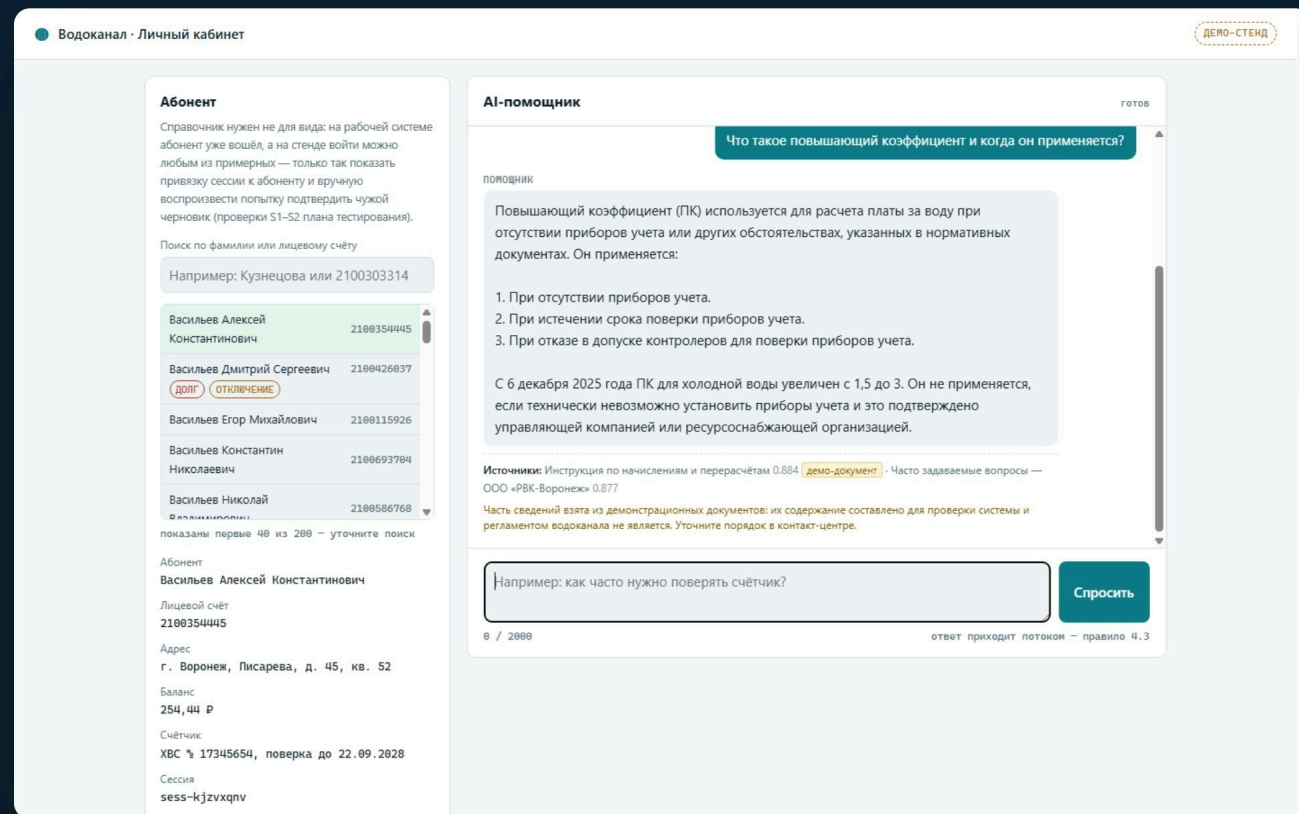
КАК ЭТО ВЫГЛЯДИТ В ИНТЕРФЕЙСЕ

Помощь в оформлении обращения — прямо в диалоге



КАК ЭТО ВЫГЛЯДИТ В ИНТЕРФЕЙСЕ

Ответ по базе знаний — с источниками, а не выдумкой



ЧТО ДАЛЬШЕ

Этап 1 – фундамент. Операции от имени абонента – следующий шаг

Сейчас · Этап 1

- Консультации и справки по данным абонента
- Помощь в подготовке обращений
- Без изменения данных абонента и без отправки заявок
- Несколько задач в одной реплике — ответ на каждую

Дальше · Этап 2

- Передача показаний, подача заявлений, регистрация обращений
- Каждое действие — только после подтверждения абонентом
- Начинается после проверки пользы и безопасности Этапа 1

Спасибо. Вопросы?

AI-помощник абонента водоканала — прототип Этапа 1, далее — MVP